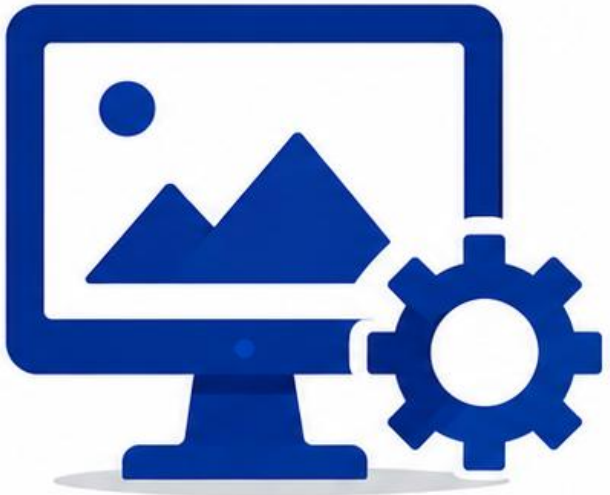
1

# Genetic Programming with Image-Related Operators and a Flexible Program Structure for Feature Learning in Image Classification



## Paper Presentation

Presented by: **James Davies**

# 2 Why Do We Need Feature Extraction?

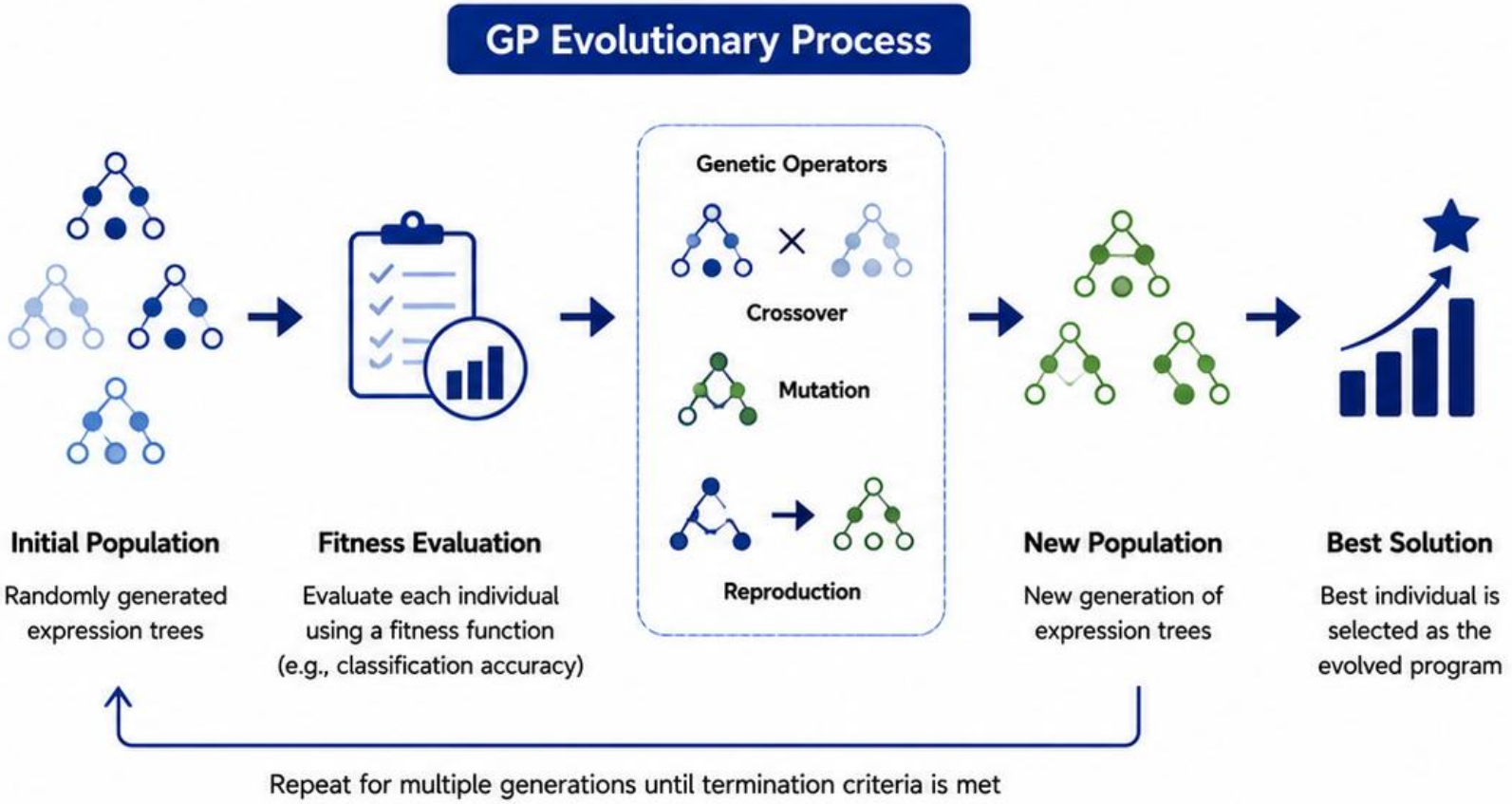
- **Computers represent images** as matrices of pixel intensities.
- **Raw pixels** contain large amounts of data with little semantic meaning.
- **Feature extraction** transforms pixels into informative and discriminative representations.
- These **feature vectors** capture the most useful information for distinguishing between classes.
- The goal is to **improve classification** accuracy while **reducing complexity**.

| Image | Raw Pixels | Feature Extraction | Feature Vector | Classification |
|---|---|---|---|---|
| Original input image | Matrix of pixel intensities | Selects and transforms the most informative information | Compact, informative representation | Used by a classifier to predict the class label |

**Key Takeaway:** Feature extraction converts raw pixel data into compact, discriminative feature vectors that improve classification performance and reduce computational complexity.

# 3 What is Genetic Programming (GP)?

- Genetic Programming is an evolutionary algorithm that **evolves computer programs or expressions** to solve problems.
- Individuals are typically represented as **expression trees** composed of functions (operators) and terminals (operands).
- A population of individuals evolves over iterations using **selection, crossover, mutation** and reproduction.
- The goal is to discover programs (feature combinations) that **maximise classification performance.**
- GP performs both **feature construction and feature selection** simultaneously.




**Key Takeaway:** GP uses evolutionary principles to automatically evolve programs (feature combinations) that solve problems. It searches a large space of possible solutions to find the fittest individuals.

# 4 How GP Represents Solutions (Expression Trees)

GP represents individuals as **expression trees**, a tree structure where each node is an operator or an operand.

Internal nodes are **functions** (operators) such as $+, -, \times, \div$, log, sqrt.

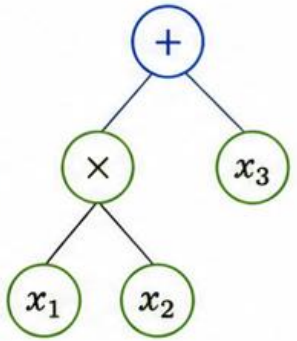
Leaf nodes are **terminals** (operands) such as variables $(x_1, x_2, \ldots, x_n)$ or constants.

Traversing the tree from the root to the leaves yields a mathematical **expression** or program.

This representation allows GP to **search and evolve complex** feature combinations automatically.

## Examples of Expression Trees

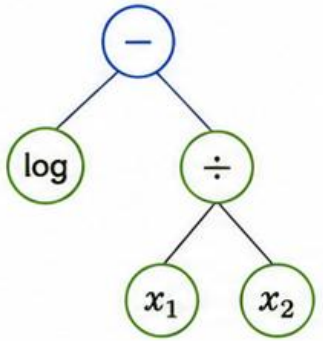
### Example 1




**Expression:**

$(x_1 \times x_2) + x_3$

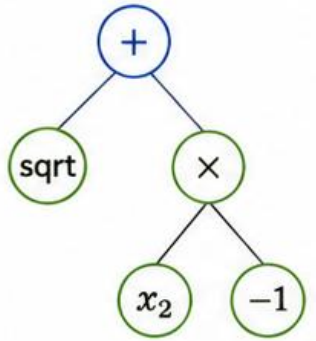
### Example 2




**Expression:**

$\log(x_1) - (x_1 \div x_2)$

### Example 3




**Expression:**

$\text{sqrt}(x_1) + (x_2 \times -1)$

### Legend

- Function (Operator)
- Terminal (Operand)
- Structure (Tree)

**Common Operators**

$+$ $-$ $\times$ $\div$ log sqrt

sin cos exp abs ...

**Key Takeaway:** GP encodes solutions as expression trees composed of functions and terminals. These trees represent mathematical expressions used to construct and select informative features.

# 5 GP Genetic Operators: Crossover, Mutation and Reproduction

**Crossover** combines parts of two parent trees to create offspring, promoting the exchange of useful sub-expressions.

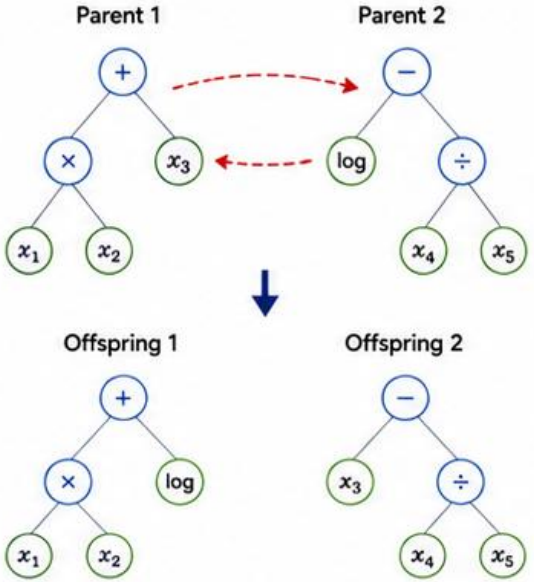
**Mutation** randomly modifies a subtree or node, introducing new structures and maintaining diversity.

**Reproduction** copies high-quality individuals unchanged to the next generation, preserving good solutions.

Together, these operators balance **exploration** and **exploitation** to evolve better **solutions** over time.

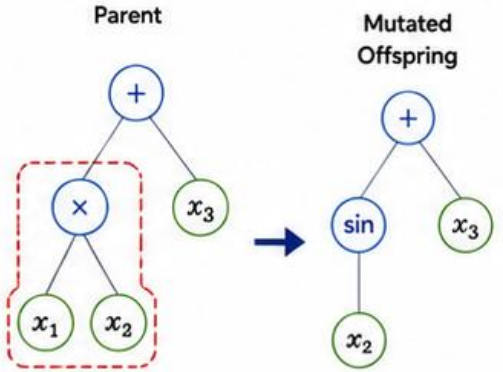
## How the Operators Work

### 1. Crossover



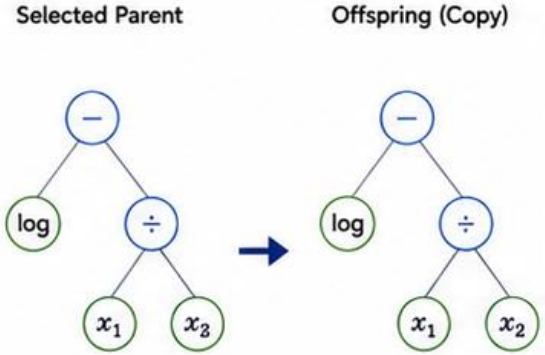

Subtrees are swapped between two parent trees.

### 2. Mutation




A randomly selected subtree is replaced or modified.

### 3. Reproduction



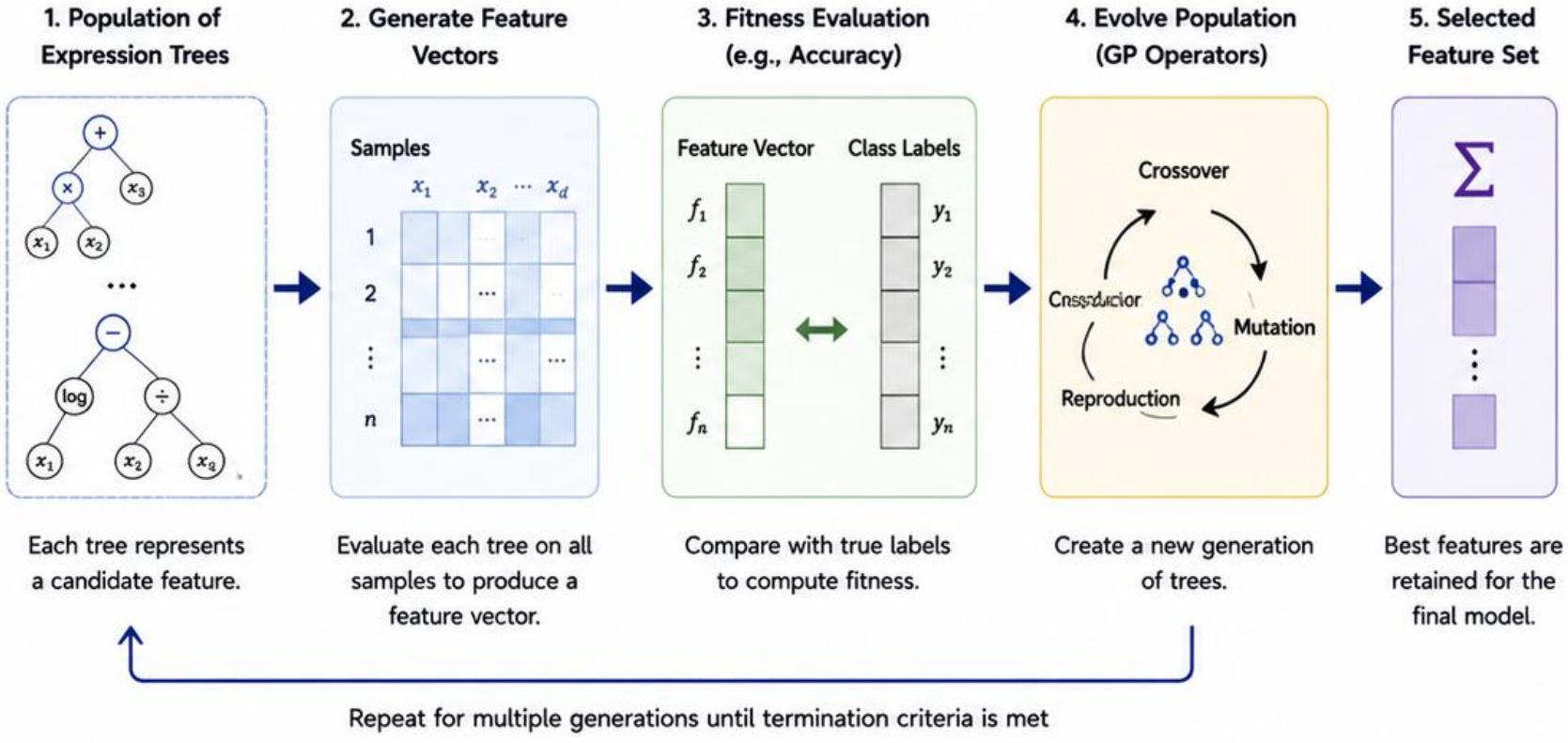

High-fitness individuals are copied unchanged.

**Key Takeaway:** Crossover recombines solutions, mutation introduces new possibilities, and reproduction preserves the best solutions. Together, these operators drive the evolution of better programs (feature combinations).

# 6 How GP Constructs and Selects Features

- **Expression trees** define mathematical transformations of the original input variables to create new features.
- **Each tree (feature)** is evaluated on the training data to produce a feature vector (one value per sample).
- A **fitness function** (e.g., classification accuracy) measures how useful each feature or feature subset is.
- **GP evolves populations** of trees, discovering combinations that improve discriminative power while reducing redundancy.
- The result is a **compact set of high-quality features** that maximise classification performance.

## GP for Feature Construction and Selection




**Key Takeaway:** GP automatically builds and selects the most informative features by evolving expression trees. These features are evaluated, refined and retained based on their ability to improve classification accuracy.

# 7 GP vs Traditional Feature Selection Methods

**Traditional methods** evaluate features individually or in small combinations.

**GP evaluates thousands of combinations simultaneously** using expression trees.

**This enables GP to discover non-linear and complex** feature interactions that others miss.

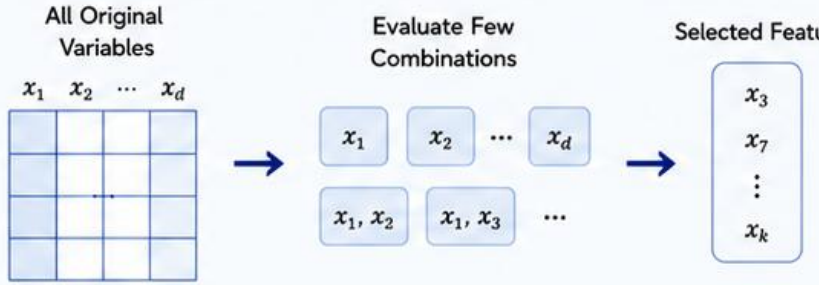
**GP produces a smaller, more informative feature set,** reducing redundancy and overfitting.

The result is **higher classification accuracy** with a more efficient and interpretable model.

## How GP Offers an Advantage

### Traditional Feature Selection

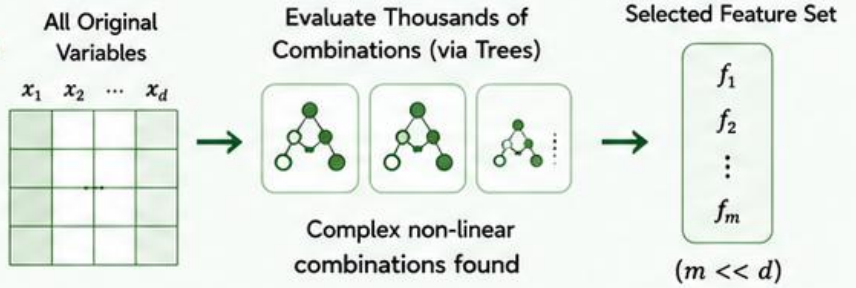
Evaluates features individually or in small subsets




- Limited search space
- Misses complex interactions
- More redundant features
- Lower classification accuracy

### GP Feature Construction and Selection

Evolves and evaluates thousands of feature combinations at once




- Explores huge search space
- Discovers complex interactions
- Fewer, more informative features
- Higher classification accuracy

**GP searches a vastly larger space of possible solutions, allowing it to evolve compact and highly discriminative feature sets that improve classification performance.**

**Key Takeaway:** GP outperforms traditional methods by exploring many more combinations and capturing complex relationships between variables, resulting in more accurate and efficient classification.

# 8 GP Pipeline for Image Classification (Our Approach)

We apply GP to **construct and select informative features** from image data for classification.

Each image is represented as a **matrix of pixel intensities** (e.g., 64×64 grayscale).

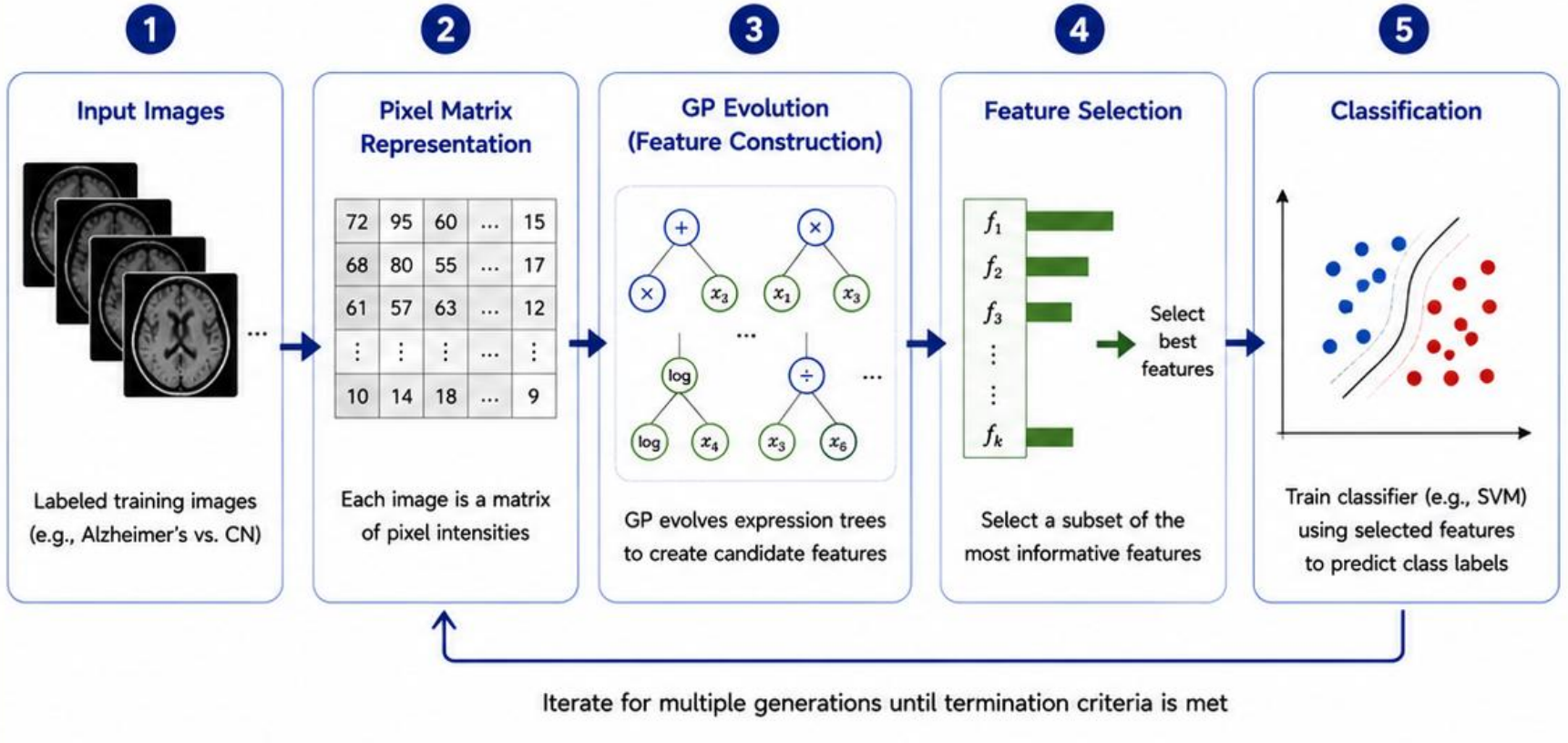
GP evolves expression trees to **create features** that transform the pixel data.

A subset of high-quality features is selected to form a compact feature vector for each image.

The selected feature vectors are used to **train a classifier** (e.g., SVM) to predict image classes.

The pipeline is evaluated on unseen test data to measure classification performance.

## End-to-End GP Pipeline




**Key Takeaway:** Our approach uses GP to automatically construct and select the most informative features from image data. These features are then used by a classifier to achieve accurate and interpretable image classification.

# 9 How Do We Know If a Tree Is Good?

*Evaluating Fitness*

Each **expression tree produces** a **feature value** for every training sample.

We compare the predicted feature values to the **true class labels**.

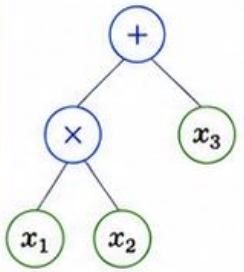
A **fitness function** measures how well the tree separates the classes (e.g., classification accuracy).

We also **penalise overly complex** trees to avoid overfitting.

Trees with **higher fitness** are more likely to be selected to create the next generation.

## How Fitness of a Tree Is Computed

### 1. Expression Tree




Example tree (candidate feature)

### 2. Compute Feature Values

| Sample | $x_1$ | $x_2$ | $x_3$ | Tree Output $f(x)$ | True Label $y$ |
|---|---|---|---|---|---|
| 1 | 0.2 | 0.5 | 1.3 | 0.61 | 0 |
| 2 | 1.1 | 0.3 | 0.8 | 1.05 | 1 |
| 3 | 0.7 | 1.2 | 0.4 | 0.96 | 1 |
| ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ |
| $n$ | 0.3 | 0.9 | 1.0 | 0.72 | 0 |

Evaluate the tree for all training samples

### 3. Measure Performance

Example: Classification Accuracy

$$\text{Accuracy} = \frac{\#\ \text{Correct Predictions}}{\text{Total Samples}}$$

(Other metrics can be used, e.g., AUC, F1-score)

Higher accuracy means a better tree

### 4. Compute Fitness

Fitness balances accuracy and complexity

$$\text{Fitness} = \text{Accuracy} - \lambda \times \text{Size}$$

where:

- Size = number of nodes
- $\lambda$ = complexity penalty (tunable parameter)

Prefer accurate trees that are also simple

**Example:**

If a tree classifies 85% of training samples correctly and has 15 nodes, with $\lambda = 0.01$:

$$\text{Fitness} = 0.85 - 0.01 \times 15 = 0.70$$

Trees with higher fitness have a better chance of being selected.

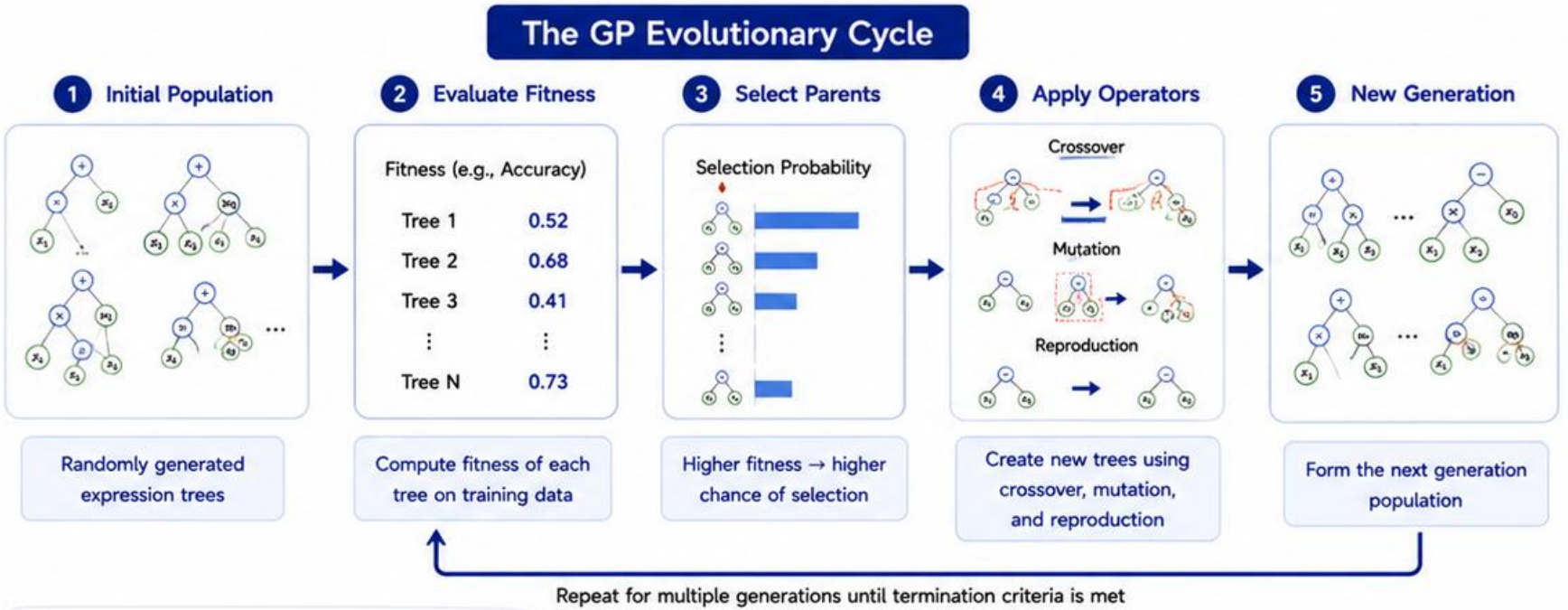
**Key Takeaway:** A tree is "good" if it produces feature values that best separate the classes on the training data while remaining simple. The fitness function rewards accuracy and penalises complexity to promote generalisable features.

# 10 How GP Evolves Better Solutions Over Time

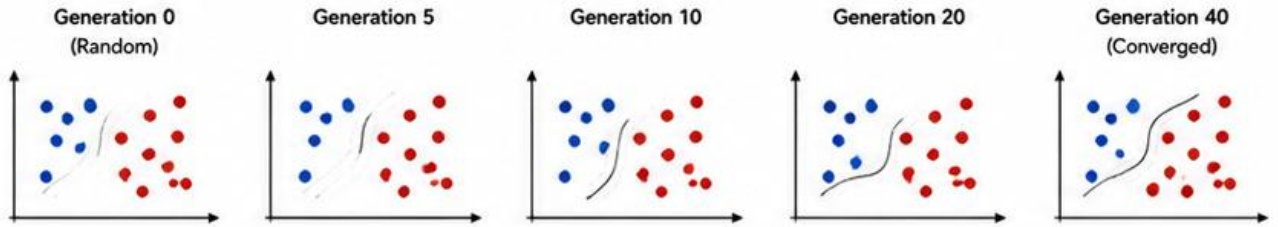
*The Evolutionary Cycle*

- GP starts with a **diverse population** of random trees.
- Trees are **evaluated** using the fitness function.
- Better trees are more likely to be **selected** for reproduction.
- Crossover, mutation and reproduction create a **new generation** of trees.
- Over successive generations, the population **improves** and converges towards high-quality solutions.
- The process continues until a **termination criterion** is met (e.g., max generations or target fitness).

## The GP Evolutionary Cycle




## Improvement Over Generations (Example)




Fitness (e.g., accuracy) typically increases with generations as GP discovers better and more informative features.

**Key Takeaway:** GP mimics natural evolution to search the space of possible solutions. By iteratively selecting, combining and refining expression trees, it progressively discovers features that lead to higher classification performance.

# 11 The Building Blocks of Evolution: GP Operators

*How New Trees Are Created*

GP uses **genetic operators** to combine and modify trees, creating new candidate solutions.

**Crossover** combines parts of two parent trees to produce offspring with traits from both.

**Mutation** randomly changes a part of a tree, introducing new structures and diversity.

**Reproduction** copies a good tree unchanged to the next generation, preserving useful solutions.

These operators balance **exploration** (searching new areas) and **exploitation** (keeping good solutions).

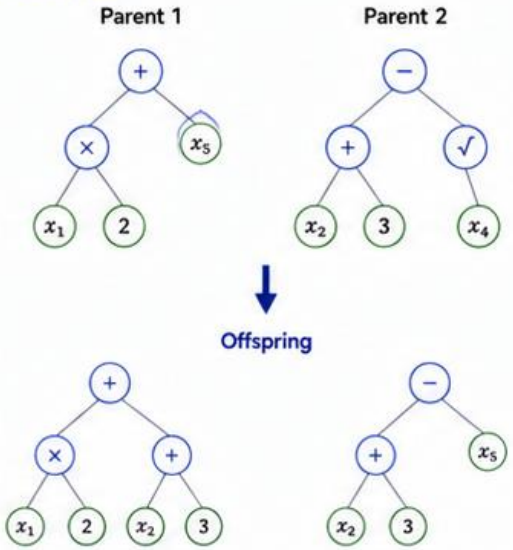
Together, they drive the evolution of **better** and **more informative** features.

## The Three GP Operators

### 1. Crossover (Recombination)

Exchange randomly selected subtrees between two parent trees.

**Example:**

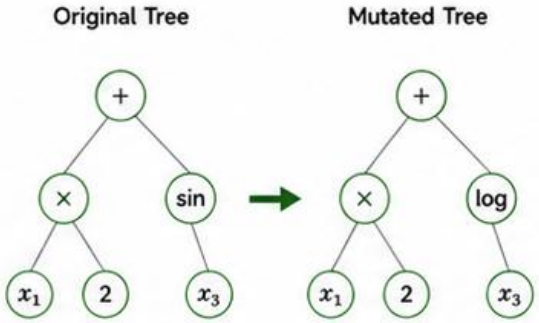



Offspring inherit structure from both parents. Promotes **combination** of useful traits.

### 2. Mutation

Randomly modify a node or subtree within a tree.

**Example:**

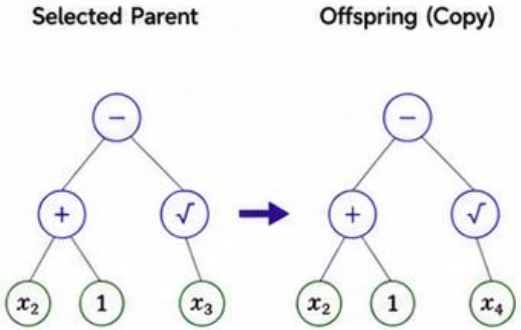



Small random changes introduce **diversity** and allow discovery of new solutions.

### 3. Reproduction (Copy)

Copy a selected tree unchanged to the next generation.

**Example:**




**Preserves** high-quality trees, ensuring good solutions are not lost.

**Key Takeaway:** Crossover, mutation and reproduction are the core mechanisms that enable GP to evolve expression trees. They create new candidate features while maintaining and improving the best solutions over generations.

# 12 From Trees to Features: Creating the Feature Vector

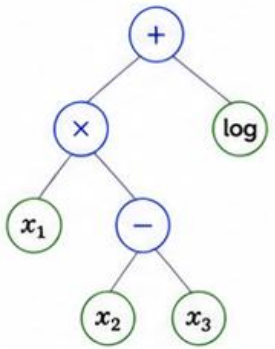
*How a Tree Becomes a Usable Feature*

- Each expression tree represents a **candidate feature**.
- For every image, the tree is evaluated to produce **one numerical value**.
- Repeating this for all training images creates a **feature vector** (one value per image).
- This vector becomes one column in the **feature matrix** used to train the classifier.
- Better trees produce feature vectors that **better separate** the classes.
- GP evolves trees that generate the **most informative** and discriminative feature vectors.

## Example: Evaluating a Tree to Create a Feature Vector

**1 Expression Tree (Candidate Feature)**

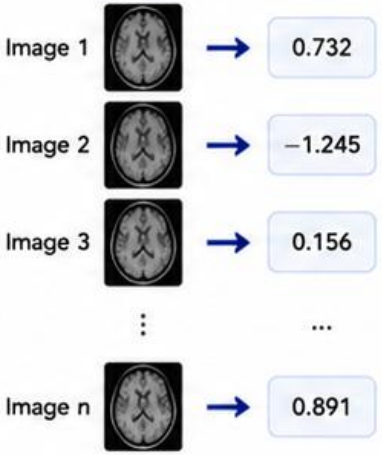
This tree defines a mathematical expression using pixel values.




Example expression:
$(x_1 \times (x_2 - x_3)) + \log(x_4)$

**2 Evaluate on Each Image**

Compute the tree's output for every image.




**3 Create Feature Vector**

Collect all outputs into a feature vector.

Feature Vector

| |
|---|
| 0.732 |
| −1.245 |
| 0.156 |
| ⋮ |
| 0.891 |

(length = $n$)

**4 Add to Feature Matrix**

This vector is one column in the feature matrix.

Feature Matrix (Example)

| Feature 1 | Feature 2 | ... | Feature $k$ |
|---|---|---|---|
| 0.245 | −1.321 | ... | 0.732 |
| −0.842 | 0.214 | ... | −1.245 |
| 0.512 | −0.753 | ... | 0.156 |
| ⋮ | ⋮ | ... | ⋮ |
| 1.102 | 0.143 | ... | 0.891 |

Each column = one GP tree (feature)
Each row = one image

This process converts an expression tree into a numeric feature that captures a specific pattern in the images.
The classifier then uses these features to learn the difference between classes.

**Key Takeaway:** Every tree produces a feature vector by being evaluated on all images. These vectors form the feature matrix used by the classifier. GP searches for trees that generate the most informative vectors for accurate classification.

# 13 From Features to Predictions: Training the Classifier

*Using GP-Derived Features to Predict Class Labels*

After GP evolution, we obtain a set of high-quality **expression trees.**

Each tree is evaluated on all training images to create a **feature vector.**

All feature vectors form the feature **matrix** (rows = images, columns = trees/features).

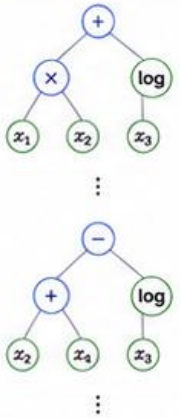
This matrix is used to **train a classifier** (e.g., SVM) with the true class labels.

The trained classifier learns to **separate the classes** in this feature space.

The classifier is then used to **predict labels** of unseen test images.

## Pipeline: From Evolved Trees to Classifier

**1 Evolved Trees (Features)**




Top $k$ evolved trees (selected features)

**2 Evaluate on All Training Images**

Images (Training) → Tree Outputs (Example)

| | Tree 1 | Tree 2 | ... | Tree $k$ |
|---|---|---|---|---|
| 1 | 0.732 | −1.321 | ... | 0.245 |
| 2 | −1.245 | 0.214 | ... | −0.842 |
| 3 | 0.156 | −0.753 | ... | 0.512 |
| ⋮ | ... | ⋮ | ... | ... |
| $n$ | 0.891 | 0.143 | ... | 1.102 |

Each cell = output of a tree (feature) on one image

**3 Feature Matrix (Design Matrix)**

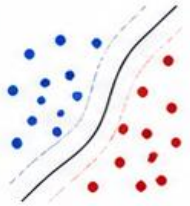
$\boldsymbol{X}$ $(n \times k)$

| | | | |
|---|---|---|---|
| 0.732 | −1.321 | ... | 0.245 |
| −1.245 | 0.214 | ... | −0.842 |
| 0.156 | −0.753 | ... | 0.512 |
| ... | ... | ... | ... |
| 0.891 | 0.143 | ... | 1.102 |

$y$ (Class Labels)

$\begin{bmatrix} 0 \\ 1 \\ 1 \\ \vdots \\ 0 \end{bmatrix}$ 0 = Class 1, 1 = Class 2 (e.g., CN vs. AD)

**4 Train Classifier (e.g., SVM)**



Train SVM using $(X, y)$ to learn the optimal decision boundary.

**5 Predict on New (Test) Images**

New Image

Feature Vector (from $k$ trees)

| | | | |
|---|---|---|---|
| 0.612 | −0.981 | ... | 0.305 |

Trained Classifier (SVM)

Predicted Label (Class 1 or Class 2)

**What Happens:**

GP provides a compact, informative feature representation of each image.

The classifier learns how to combine these features to make accurate predictions.

**Benefits:**

- GP discovers discriminative features automatically
- Classifier handles the final decision boundary
- Improves generalisation on unseen data

**Key Takeaway:** The evolved trees are transformed into numeric features that capture important patterns in the images. A classifier (e.g., SVM) is trained on these features to learn the best way to separate the classes and make predictions on new, unseen images.

# 14 Why Some Trees (Features) Matter More Than Others

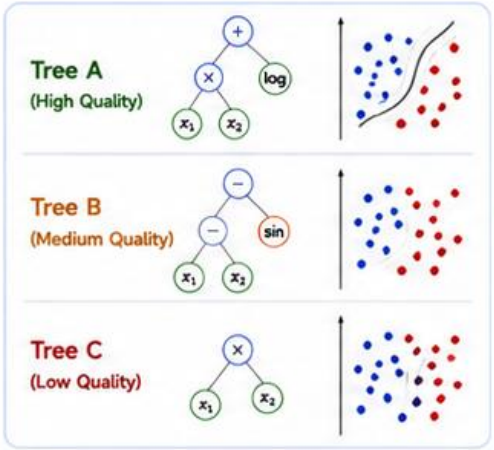
*Feature Importance in GP*

- Not all trees contribute **equally** to classification performance.
- Trees that produce more **discriminative feature vectors** lead to better class separation.
- These trees achieve **higher fitness** (e.g., higher accuracy).
- Higher fitness increases the chance of **selection** for the next generation.
- Over generations, GP favours trees that capture the **most informative patterns** in the data.
- The final feature set is a subset of **high-quality, complementary** trees.

## How Feature Importance Emerges in GP

### 1. Different Trees, Different Quality

Trees produce feature vectors with varying ability to separate classes.




Better separation ⇒ higher quality.
Poor separation ⇒ lower quality.

### 2. Evaluated by Fitness

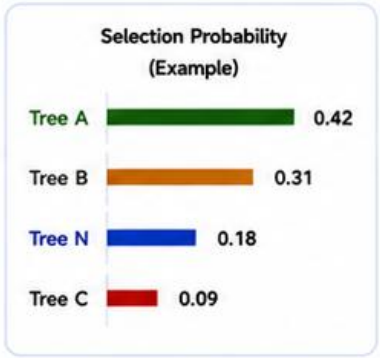
Each tree is evaluated on the training data.

| Tree | Fitness (Accuracy) |
|---|---|
| Tree A | 0.92 |
| Tree B | 0.68 |
| Tree C | 0.28 |
| ... | ⋮ |
| Tree N | 0.75 |

Higher accuracy = higher fitness.

### 3. Higher Fitness ⇒ Higher Chance of Selection

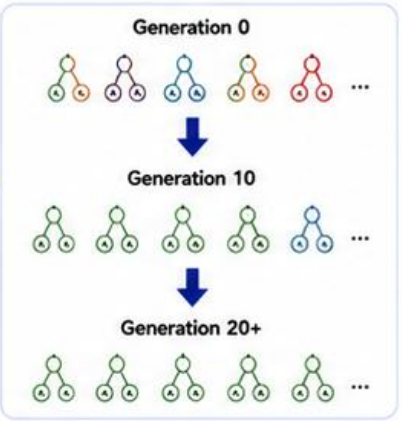
Selection probability is proportional to fitness.




Better trees are more likely to become parents.

### 4. Over Generations Good Trees Dominate

GP evolves towards a population rich in high-quality trees.




The population converges to the most informative features.

In GP, importance is not assigned explicitly— it **emerges naturally**.
Trees that consistently help the classifier perform well are rewarded with higher fitness and survive.
As a result, the final model is built from the most important and discriminative features.

**Analogy:**
Like natural selection in biology, the fittest trees (features) thrive, while weaker ones disappear over time.

**Key Takeaway:** Some trees matter more because they produce features that better separate the classes, leading to higher fitness. GP uses this feedback to prefer those trees, ensuring the final feature set is informative, compact and effective for classification.

# 15 Putting It All Together: The GP-Based Feature Learning Pipeline

*From Raw Images to Accurate Classification*

**Start with raw images.**
Preprocess the data and split into training and test sets.

**GP evolves expression trees.**
Using crossover, mutation and reproduction to create new trees.

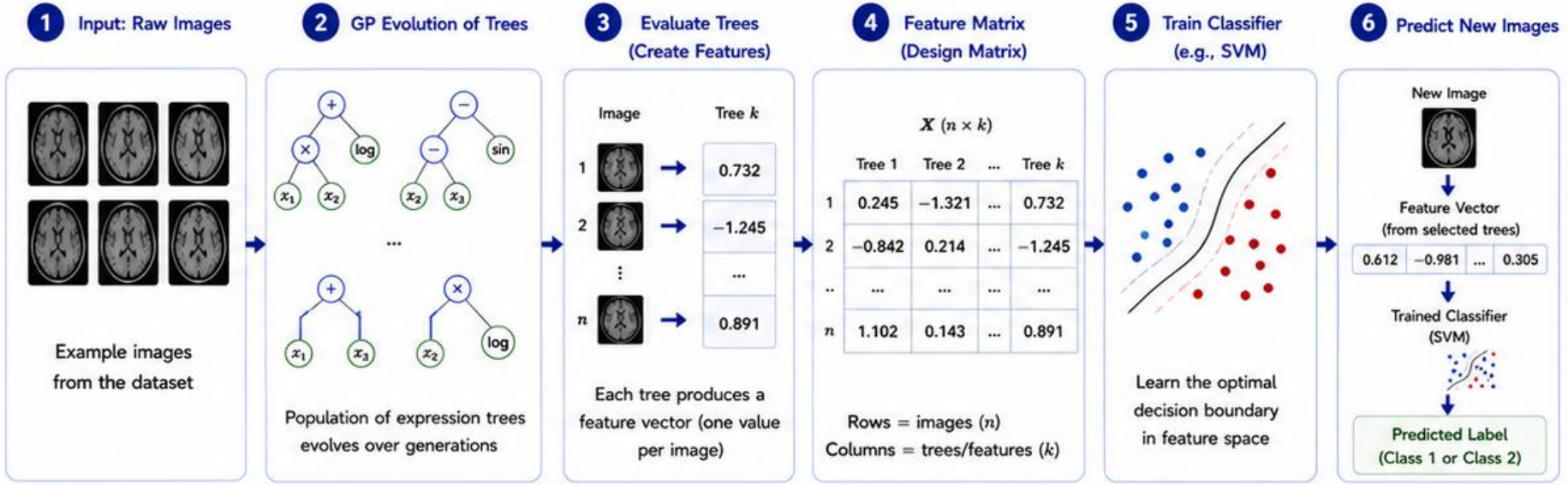
**Trees are evaluated to create features.**
Each tree generates a feature vector (one value per image).

**Build the feature matrix.**
All feature vectors form the design matrix used for model training.

**Train a classifier.**
A model (e.g., SVM) learns the best decision boundary in this space.

**Make predictions on new data.**
The trained model uses GP-derived features to predict unseen images.

## End-to-End GP Feature Learning Pipeline




## What Makes This Powerful?

**Automatic feature discovery**
GP finds informative features without manual design.

**Compact and interpretable**
Final model uses a small set of highly discriminative trees.

**High performance**
Better class separation leads to higher classification accuracy.

**General and flexible**
Works with different datasets, classifiers and domains.

**Robust and efficient**
Good generalisation to unseen data with fewer features.

**Key Takeaway:** GP automatically evolves expression trees that act as powerful, problem-specific features. These features are combined to train a classifier that can accurately predict class labels on new, unseen images. **This end-to-end pipeline turns raw image data into accurate predictions through evolutionary intelligence.**

# 16 Summary: How GP Learns Powerful Features for Classification

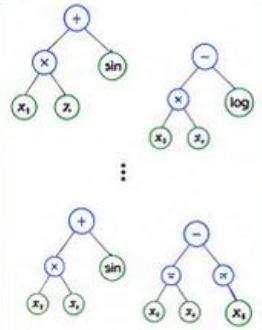
*From Pixels to Predictions via Evolutionary Intelligence*

- GP represents candidate solutions as **expression trees** built from functions and terminals.
- Through **crossover**, **mutation** and **reproduction**, trees evolve over generations.
- Each tree is evaluated on the training images to create a **feature vector**.
- All feature vectors form the **feature matrix** used to train a classifier (e.g., SVM).
- GP favours trees that produce more **discriminative features**, improving classification.
- The final model uses the most informative features to make **accurate predictions** on new, unseen images.

## Summary of the GP Feature Learning Algorithm

**1 Initialise Population**

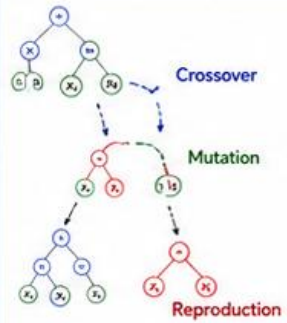
Create a population of random expression trees.



Population of random trees

**2 Evolve Trees**

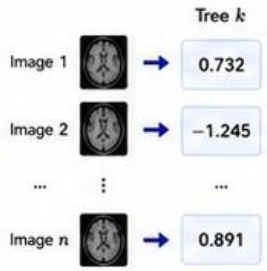
Apply crossover, mutation and reproduction to evolve trees over generations.




Evolution creates new and better trees

**3 Evaluate Trees (Create Features)**

Evaluate each tree on all training images to create a feature vector.




Each tree → one feature (one value per image)

**4 Build Feature Matrix**

Combine all feature vectors to form the design matrix.

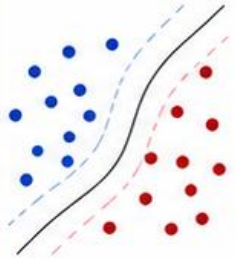
$X$ $(n \times k)$

| | Tree 1 | Tree 2 | ... | Tree $k$ |
|---|---|---|---|---|
| 1 | 0.245 | −1.321 | ... | 0.732 |
| 2 | −0.842 | 0.214 | ... | −1.245 |
| ... | ... | ... | ... | ... |
| $n$ | 1.102 | 0.143 | ... | 0.891 |

Rows = images ($n$)
Columns = trees/features ($k$)

**5 Train Classifier**

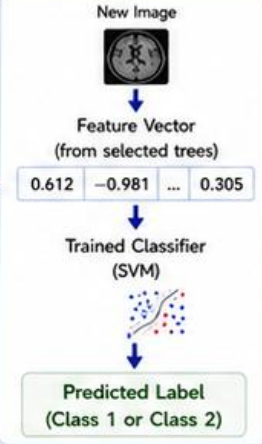
Train a classifier (e.g., SVM) using the feature matrix and class labels.



Learn the optimal decision boundary in feature space

**6 Predict New Images**

Use the trained model to predict labels of unseen images.




## What We Achieve with GP-Based Feature Learning

**Automatic Discovery**
GP automatically discovers informative features without manual feature engineering.

**Compact Representation**
Produces a small set of highly discriminative features that capture important patterns.

**Better Performance**
Improves class separation and leads to higher classification accuracy.

**Generalisation**
Works well on new, unseen data and across different datasets.

**Efficiency**
Reduces dimensionality, training time and memory requirements.

**Key Takeaway:** Genetic Programming evolves expression trees that are turned into powerful, discriminative features.
These features, used to train a classifier, enable accurate and robust classification of medical images.
This framework combines evolutionary search with machine learning to solve complex real-world problems.

# 17 Results in Action: What GP Learns and Why It Works

*Discovering Discriminative Features That Drive Accurate Predictions*

**GP discovers meaningful image patterns.**
Expression trees capture combinations of pixel intensities that highlight important structures.

**These patterns become powerful features.**
The features form a compact representation that separates the classes effectively.

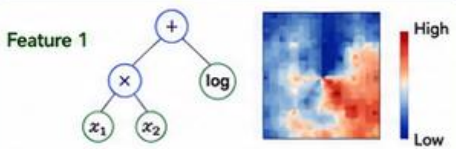
**A classifier learns the best decision boundary.**
In this space, classes are more separable, leading to higher accuracy.

**Better performance on unseen data.**
The model generalises well to new images, not just the training data.

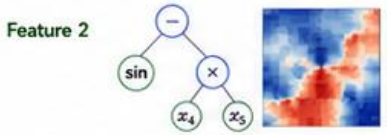
**Why it works.**
GP performs automated feature engineering tailored to the data and the task.

## Example Results from a GP-Based Feature Learning Model

### 1. Example Evolved Trees (Features)




$$f_1 = (x_1 \times x_2) + \log(x_3)$$

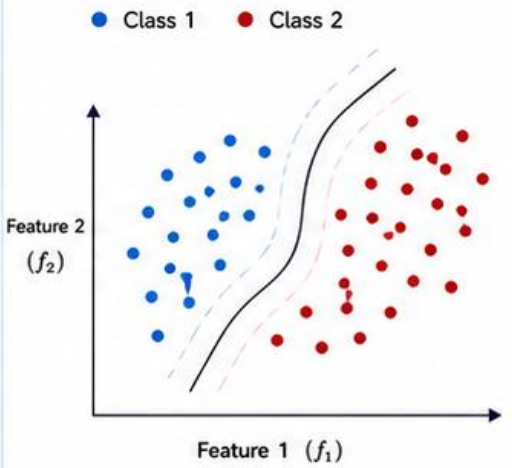



$$f_2 = \sin(x_4) - (x_5 \times x_6)$$


$$f_3 = \frac{x_7}{x_8} + \log(x_9)$$

Trees produce features that focus on informative regions and relationships.

### 2. Feature Space ( after GP Feature Learning)




In the learned feature space, the classes are more separable. (SVM finds the optimal boundary)

### 3. Performance (Example)

| Dataset | Accuracy (%) | F1-Score (%) |
|---|---|---|
| Training Set | 98.7 | 98.6 |
| Test Set | 96.2 | 96.1 |
| Unseen Set | 94.8 | 94.6 |

**Confusion Matrix (Test Set)**

| | | Predicted | |
|---|---|---|---|
| | | Class 1 | Class 2 |
| Actual | Class 1 | 482 | 18 |
| | Class 2 | 24 | 476 |

High accuracy and balanced results across classes.

### 4. What the Model Learns

**Detects subtle patterns** that humans may miss, e.g., texture, edges, and intensities.

**Reduces complexity** by selecting a small set of highly informative features.

**Improves generalisation** by focusing on patterns that are consistent across data.

**Delivers robust classification** even on noisy or unseen data.

GP automatically learns and selects the features that best explain the differences between classes. These features drive the classifier to make accurate, reliable predictions on new, unseen images.

**Key Takeaway:** GP evolves expression trees that become high-quality, discriminative features. These features create a separable feature space where a classifier can learn the optimal boundary, **resulting in accurate and generalisable predictions on new images.**